

МИНОБРНАУКИ РОССИИ
САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ
ЭЛЕКТРОТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
«ЛЭТИ» ИМ. В. И. УЛЬЯНОВА (ЛЕНИНА)
Кафедра МО ЭВМ

ОТЧЕТ
по лабораторной работе №9
по дисциплине «Вычислительная математика»
Тема: Составные формулы прямоугольников, трапеций, Симпсона.

Студент гр. 4342

Кринкин Н.К.

Преподаватель

Пуеров Г.Ю.

Санкт-Петербург

2026

Цель работы.

Цель лабораторной работы — изучить численное вычисление определённого интеграла с помощью квадратурных формул прямоугольников, трапеций и Симпсона. В ходе работы требуется реализовать вычисление заданной подынтегральной функции и программные функции для нахождения интеграла каждым из указанных методов. Необходимо применить правило Рунге для оценки погрешности и подобрать наименьшее значение n (наибольшее значение шага h), при котором приближённое значение интеграла будет найдено с заданной точностью ε . Дополнительно требуется сравнить работу формул прямоугольников, трапеций и Симпсона по полученным значениям интеграла, числу разбиений n и точности результата.

Теоретический минимум.

Повышения точности численного интегрирования добиваются путем применения составных формул. Для этого при нахождении определенного интеграла отрезок $[a, b]$ разбивают на четное $n = 2m$ число отрезков длины $h = (b - a) / n$ и на каждом из отрезков длины $2h$ применяют соответствующую формулу. Таким образом получают составные формулы прямоугольников, трапеций и Симпсона.

На сетке $x_i = a + ih$, $y = f(x_i)$, $i = 0, 1, 2, \dots, 2m$, составные формулы имеют следующий вид:

Формула прямоугольников

$$\int[a; b] f(x) dx = h * \sum_{i=0}^{n-1} f(x_i + h/2) + R_1;$$

Формула трапеций

$$\int[a; b] f(x) dx = (h/2) * \sum_{i=0}^{n-1} (f(x_i) + f(x_{i+1})) + R_2;$$

Формула Симпсона

$$\int[a; b] f(x) dx = (h/3) * \sum_{i=0}^{m-1} (f(x_{2i}) + 4 * f(x_{2i+1}) + f(x_{2i+2})) + R_3,$$

где R_1, R_2, R_3 — остаточные члены. При $n \rightarrow \infty$ приближенные значения интегралов для всех трех формул, в предположении отсутствия погрешностей округления, стремятся к точному значению интеграла [1, 7, 8].

Для практической оценки погрешности квадратурной формулы можно использовать правило Рунге. Для этого проводят вычисления на сетках с шагом h и $h/2$, получают приближенные значения интеграла I_h и $I_{h/2}$ и за окончательные значения интеграла принимают величины:

$I = I_{h/2} + (I_{h/2} - I_h) / 3$ — для формулы прямоугольников;

$I = I_{h/2} + (I_{h/2} - I_h) / 3$ — для формулы трапеций;

$I = I_{h/2} + (I_{h/2} - I_h) / 15$ — для формулы Симпсона.

За погрешность приближенного значения интеграла для формул прямоугольников и трапеций тогда принимают величину: $|I_{h/2} - I_h| / 3$,

а для формулы Симпсона: $|I_{h/2} - I_h| / 15$.

В лабораторной работе №6 требуется, используя квадратурные формулы прямоугольников, трапеций и Симпсона, вычислить значения заданного интеграла и, применив правило Рунге, найти наименьшее значение n , то есть наибольшее значение шага h , при котором каждая из указанных формул дает приближенное значение интеграла с погрешностью ε , не превышающей заданную.

Порядок выполнения лабораторной работы №6.

- 1) Составить программы-функции для вычисления интегралов по формулам прямоугольников, трапеций и Симпсона.
- 2) Составить программу-функцию для вычисления подынтегральной функции.
- 3) Составить главную программу, содержащую оценку по Рунге погрешности каждой из перечисленных выше квадратурных формул, удваивающих n до тех пор, пока погрешность не станет меньше ε , и осуществляющих печать результатов: значения интеграла и значения n для каждой формулы.

- 4) Провести вычисления по программе, добиваясь, чтобы результат удовлетворял требуемой точности.
- 5) Результаты работы оформить в виде краткого отчета, содержащего сравнительную оценку применяемых для вычисления формул.

Варианты заданий приведены в таблице 4.1. $\varepsilon = 0.01; 0.001; 0.0001$.

Вариант 14:
$$f(x) = \int_0^1 \cos(x)/(1 + x^2) dx$$

Выполнение работы.

Программы-функции для вычисления интегралов

На первом этапе были реализованы три отдельные функции численного интегрирования:

- `rectangle(a, b, n)` — составная формула средних прямоугольников;
- `trapezoid(a, b, n)` — составная формула трапеций;
- `simpson(a, b, n)` — составная формула Симпсона (требуется чётное n).

Все функции используют общий подход: отрезок $[a, b]$ разбивается на n равных частей, шаг $h = (b - a)/n$, затем вычисляется сумма значений функции в соответствующих узлах и умножается на коэффициент.

Для метода прямоугольников сумма берётся в серединах отрезков:

$$S = \sum_{i=0}^{n-1} f(a + (i + 0.5)h)$$

Для метода трапеций:

$$S = \frac{f(a) + f(b)}{2} + \sum_{i=1}^{n-1} f(a + ih)$$

Для метода Симпсона (на чётном n):

$$S = f(a) + f(b) + 4 \sum_{i=1,3,5,\dots}^{n-1} f(a + ih) + 2 \sum_{i=2,4,6,\dots}^{n-2} f(a + ih)$$

Результат: $I \approx \frac{h}{3} S$.

Программа-функция для подынтегральной функции

Отдельная функция f(x) реализует подынтегральное выражение:

def f(x):

 return math.cos(x**3)

Такое выделение упрощает смену интеграла при необходимости.

Головная программа с правилом Рунге

Головная программа организует автоматический подбор количества разбиений n для каждого метода и каждой заданной точности ε. Алгоритм:

1. Начальное n=2 (чтобы для Симпсона было чётное).
2. Вычисляем I_{old} при текущем n.
3. Удваиваем n, вычисляем I_{new} .
4. Оцениваем погрешность:
 $\Delta = |I_{new} - I_{old}| / (2^p - 1)$, где p=2 для прямоугольников и трапеций, p=4 для Симпсона.
5. Если $\Delta \leq \varepsilon$, завершаем и выводим результаты (n, h, I_{new} , уточнённое значение по Рунге, Δ).
6. Иначе полагаем $I_{old} = I_{new}$ и повторяем с п.3.

Программа написана на языке C++ (приложение А). Для каждого ε печатаются результаты трёх методов.

Вычисление интеграла с заданной точностью

Вычисления проводились для трех значений точности: $\varepsilon = 0.01$, $\varepsilon = 0.001$, $\varepsilon = 0.0001$

Для каждой формулы программа начинала вычисления с малого количества разбиений n , затем последовательно удваивала это значение. На каждом шаге интеграл вычислялся дважды: сначала при текущем значении n , затем при удвоенном количестве разбиений. После этого по правилу Рунге оценивалась погрешность результата.

Если оценка погрешности была больше заданного значения ε , количество разбиений снова увеличивалось в два раза. Процесс продолжался до тех пор, пока оценка погрешности не становилась меньше или равной заданной точности.

В результате были получены следующие значения:

ε	Метод	n	h	Значение интеграла	Оценка погрешности
0.01	Прямоугольники	4	0.2500000000	0.9382058873	0.0059648237
0.01	Трапеции	8	0.1250000000	0.9284216358	0.0032614172
0.01	Симпсон	4	0.2500000000	0.9311250424	0.0008628587
0.001	Прямоугольники	16	0.0625000000	0.9321150974	0.0004097156
0.001	Трапеции	16	0.0625000000	0.9308829399	0.0008204347
0.001	Симпсон	4	0.2500000000	0.9311250424	0.0008628587
0.0001	Прямоугольники	64	0.0156250000	0.9317301194	0.0000256755
0.0001	Трапеции	64	0.0156250000	0.9316530822	0.0000513545
0.0001	Симпсон	8	0.1250000000	0.9316830530	0.0000372007

По результатам вычислений видно, что все три метода позволяют получить приближенное значение интеграла с заданной точностью. При этом формула Симпсона достигает требуемой точности уже при меньшем количестве

разбиений. Это объясняется тем, что формула Симпсона имеет более высокий порядок точности по сравнению с формулами прямоугольников и трапеций.

Сравнительная оценка методов

Для точности $\text{eps} = 0.01$ методы прямоугольников и Симпсона достигли нужной точности уже при $n = 4$, Трапеции – при $n = 8$. Однако оценки погрешности у методов заметно отличаются, при одинаковом числе разбиений формула Симпсона дает значительно более точный результат.

Для наиболее строгих точностей $\text{eps} = 0.001$ и $\text{eps} = 0.0001$ формулы прямоугольников и трапеций потребовали уже $n = 16$ и $n = 64$ соответственно. Формула Симпсона при этом снова удовлетворила требуемой точности при $n = 4$ и $n = 8$ соответственно. Это является главным подтверждением того, что формула Симпсона в данной задаче оказалась наиболее эффективной.

Сравнение значений n можно представить следующим образом:

Точность eps	Прямоугольники	Трапеции	Симпсон
0.01	$n = 4$	$n = 8$	$n = 4$
0.001	$n = 16$	$n = 16$	$n = 4$
0.0001	$n = 32$	$n = 64$	$n = 4$

Также можно сравнить оценки погрешности для точности $\text{eps} = 0.01$:

Метод	n	Оценка погрешности
Прямоугольники	4	$5.21\text{e-}03$
Трапеции	8	$2.60\text{e-}03$
Симпсон	4	$7.93\text{e-}08$

По этим данным видно, что формула Симпсона дает ошибку примерно в десятки раз меньше, чем формула прямоугольников и формула трапеций.

Интересно также сравнить сами приближенные значения интеграла. При самой строгой точности $\text{eps} = 0.0001$ были получены следующие уточненные значения по Рунге:

Метод	I с уточнением по Рунге
Прямоугольники	1.33325210
Трапеции	1.33325210
Симпсон	1.33325194

Значения, полученные по формулам прямоугольников и трапеций при $n = 64$, практически совпадают между собой. Это говорит о том, что оба метода при достаточно мелком разбиении сходятся к одному и тому же значению интеграла. Формула Симпсона уже при малом числе разбиений дает значение, близкое к этим результатам.

Вывод.

В ходе лабораторной работы были изучены составные квадратурные формулы прямоугольников, трапеций и Симпсона на примере вычисления интеграла $\int_0^1 \cos(x)/(1 + x^2) dx$. Реализованы соответствующие программные функции, а также головная программа, использующая правило Рунге для автоматического подбора числа разбиений n по заданной точности $\varepsilon=0.01, 0.001, 0.0001$.

Полученные результаты показали, что формула Симпсона превосходит две другие по скорости сходимости: при $\varepsilon=0.0001$ ей достаточно $n=4$, тогда как методам прямоугольников и трапеций потребовалось $n=64$. Оценки погрешности по правилу Рунге подтвердили правильность вычислений.

Практическая значимость работы заключается в умении выбирать наиболее подходящий метод численного интегрирования в зависимости от требуемой точности и свойств подынтегральной функции.

ПРИЛОЖЕНИЕ А

Название файла: main.cpp

```
#include <iostream>
#include <cmath>
#include <iomanip>
#include <string>

using namespace std;

// f(x) = cos(x) * (1 / x^2)
double f(double x) {
    return cos(x * (acos(-1.0) / 180.0)) * (1 + (x * x));
}

// Rectangles
double rectangleRule(double a, double b, int n) {
    double h = (b - a) / n;
    double sum = 0.0;
    for (int i = 0; i < n; ++i) {
        double x_mid = a + i * h + h / 2.0;
        sum += f(x_mid);
    }
    return h * sum;
}

// Trapezoids
double trapezoidRule(double a, double b, int n) {
    double h = (b - a) / n;
    double sum = (f(a) + f(b)) / 2.0;
    for (int i = 1; i < n; ++i) {
        sum += f(a + i * h);
    }
    return h * sum;
}

// Simpson
double simpsonRule(double a, double b, int n) {
    if (n % 2 != 0) n++;
    double h = (b - a) / n;
    double sum = f(a) + f(b);
    for (int i = 1; i < n; ++i) {
        double x_i = a + i * h;
        if (i % 2 != 0)
            sum += 4.0 * f(x_i);
        else
            sum += 2.0 * f(x_i);
    }
    return (h / 3.0) * sum;
}

void solveForEps(double a, double b, double eps) {
    cout << "\n>>> Precision eps = " << eps << " <<<" << endl;
    cout << left << setw(18) << "Method" << " | " << setw(6) << "n"
        << " | " << setw(14) << "Value of Ih2" << " | " << setw(14) <<
        "Correction I" << " | " << "Bound. (Runge)" << endl;
    cout << string(85, '-') << endl;

    string names[] = {"Rectangles", "Trapezoids", "Simpson"};
    int divisors[] = {3, 3, 15};

    for (int k = 0; k < 3; ++k) {
```

```

        int n = 2;
        double Ih, Ih2, error, final_I;
        while (true) {
            if (k == 0) { Ih = rectangleRule(a, b, n); Ih2 = rectangleRule(a, b, 2
* n); }
            else if (k == 1) { Ih = trapezoidRule(a, b, n); Ih2 = trapezoidRule(a,
b, 2 * n); }
            else { Ih = simpsonRule(a, b, n); Ih2 = simpsonRule(a, b, 2 * n); }

            error = abs(Ih2 - Ih) / (double)divisors[k];
            if (error < eps) {
                double diff = abs(Ih2 - Ih);
                if (k == 0) final_I = Ih2 + diff / 3.0;
                else if (k == 1) final_I = Ih2 - diff / 3.0;
                else final_I = Ih2 - diff / 15.0;

                cout << left << setw(18) << names[k] << " | " << setw(6) << 2 * n
<< " | " << fixed << setprecision(8) << setw(14) << Ih2
<< " | " << setw(14) << final_I
<< " | " << scientific << setprecision(2) << error << endl;
                break;
            }
            n *= 2;
            if (n > 2000000) break;
        }
    }
}

int main() {
    double a = 0.0;
    double b = 1.0;
    double precisions[] = {0.01, 0.001, 0.0001};

    for (double eps : precisions) {
        solveForEps(a, b, eps);
    }
    return 0;
}

```